



Sybil: Project Proposal

Vulnerability detection framework, using a neurosymbolic approach, integrating static analysis tools (SAST) and large language models (LLM).

[Introduzione](#)

[Related works](#)

[Metodologia](#)

[1. Analisi dei Requisiti](#)

[1.1 Requisiti funzionali - iter 0](#)

[1.2 Requisiti funzionali - iter 1-4](#)

[1.2 Requisiti non funzionali](#)

[2. Use Case](#)

[3. Sequence Diagram — analisi di una repo \(UC1\)](#)

[Architettura](#)

[Evoluzione del sistema](#)

[Pipeline a tre agenti](#)

[Loop agentico — ReAct](#)

[Tool MCP e template system](#)

[VulnFamilyProfile](#)

[Sandbox e isolamento](#)

[Pattern architetturali](#)

[Component diagram](#)

[Sequence diagram — pipeline 3 agenti](#)

[Class / Component Diagram](#)

[5. Package Diagram](#)

Introduzione

Il rilevamento automatico di vulnerabilità nel codice sorgente è un problema

aperto nella sicurezza del software. Gli strumenti di analisi statica come CodeQL permettono di specificare pattern di vulnerabilità come query su rappresentazioni strutturate del codice, e sono adottati su larga scala nell'industria. Tuttavia, la loro efficacia dipende in modo critico dalla qualità delle *specifiche di taint*: definizioni manuali di quali API costituiscono una sorgente di dati non attendibili, un punto di sink pericoloso o un sanitizer valido. Queste specifiche richiedono competenze trasversali in sicurezza, program analysis e richiede di conoscere le librerie specifiche del linguaggio, e la loro cura è un processo costoso e soggetto a errori (error-prone). Per linguaggi come Python e C, tali specifiche sono frammentarie o assenti, lasciando ampie classi di vulnerabilità sistematicamente scoperte dagli strumenti esistenti.

Un approccio naturale per ridurre questo onere è l'impiego di Large Language Model (LLM), che mostrano capacità promettenti nel ragionamento semantico sul codice. Lavori recenti come IRIS, QLPro e QLCoder esplorano l'integrazione tra LLM e analisi statica per inferire automaticamente specifiche di taint o generare query CodeQL. Questi sistemi ottengono risultati significativi, ma condividono tre limitazioni strutturali. In primo luogo, si rivolgono esclusivamente a Java, trascurando ecosistemi come Python e C per cui non esiste ancora un benchmark neuro-simbolico equivalente. In secondo luogo, richiedono LLM commerciali di grandi dimensioni (GPT-4o, Claude Sonnet, Claude-3.7-thinking) rendendo il costo computazionale e la dipendenza da servizi cloud una barriera all'adozione. In terzo luogo, trattano il risultato dell'analisi statica come verdetto finale oppure utilizzando solo paradigmi LLM-as-Judge, senza un meccanismo di conferma dinamica dell'effettiva sfruttabilità della vulnerabilità rilevata.

Related works

Ziyang et al. [IRIS, 2024] propongono un approccio neuro-simbolico in cui il LLM inferisce specifiche di taint CWE-specifiche analizzando le API di librerie third-party, eliminando la necessità di definirle manualmente a priori. Le specifiche vengono poi utilizzate da CodeQL per eseguire la taint analysis, e i risultati delle query vengono restituiti al LLM insieme al contesto del codice per una valutazione finale dei findings. IRIS dimostra che l'integrazione LLM-analisi statica migliora significativamente la copertura rispetto a CodeQL standalone, rilevando 55 vulnerabilità su 120 contro le 27 di CodeQL su CWE-Bench-Java.

Tuttavia, il sistema opera esclusivamente su Java, richiede un LLM commerciale (GPT-4), e tratta il risultato di CodeQL come verdetto finale, senza conferma dinamica della sfruttabilità.

Li et al. [MoCQ, 2025] propongono un framework in cui il LLM genera query CodeQL e Joern partendo da esempi di codice vulnerabile, guidato da un sottoinsieme estratto automaticamente del DSL dello strumento. Un validatore simbolico trace-driven esegue le query e fornisce feedback fine-grained al LLM per iterare fino alla correzione. MoCQ supporta PHP e JavaScript, richiede esempi di vulnerabilità noti come input, e usa LLM commerciali con un budget fino a 1000 iterazioni per query.

Gu et al. [QLPro, 2025] adottano un approccio a triplo voto per classificare source, sink e sanitizer, seguito da un ciclo Writer/Executor/Repairer in cui il LLM genera, esegue e corregge la query CodeQL. Applicato a Java con Claude-3.7-thinking, raggiunge il 66.1% di rilevamento contro il 38.7% di CodeQL standalone.

Wang et al. [QLCoder, 2025] costruiscono un framework agentico che sintetizza query CodeQL direttamente da metadati CVE, sfruttando un'interfaccia MCP con Language Server Protocol per la guida sintattica e un database RAG per il recupero semantico di query esistenti e documentazione. Valutato su 176 CVE Java, raggiunge un F1 di 0.70 contro 0.073 di CodeQL. Come IRIS, il sistema è Java-only e richiede LLM commerciali.

In tutti e tre i casi il LLM produce CodeQL direttamente — la difficoltà del DSL è affrontata con più potenza computazionale o più iterazioni, non con una decomposizione che la elimini dal task del modello.

Sajadi et al. [AXE, 2026] presentano un framework multi-agente per la conferma dinamica di vulnerability report, operando in un regime grey-box: l'input è il minimo disponibile dall'output di un tool statico — una classificazione CWE e una posizione nel codice sorgente. Un agente Strategist formula ipotesi di sfruttabilità, un Explorer recupera contesto dal sorgente on-demand, e un Exploiter esegue tentativi concreti contro il servizio target in ambiente containerizzato. Il processo produce un PoC riproducibile con oracle di verifica. Valutato su CVE-Bench, raggiunge il 30% di success rate, tre volte superiore ai baseline black-box.

AXE dimostra che integrare metadata dell'analisi statica nella fase di exploitation migliora significativamente l'efficacia rispetto all'approccio puramente black-box, ma si limita ad applicazioni web con superficie HTTP e richiede GPT-4o.

Questa tesi presenta **Sibyl**, un agente di sicurezza neuro-simbolico che affronta questi tre problemi attraverso una ridefinizione del ruolo di CodeQL nella pipeline. L'intuizione centrale è che CodeQL non debba essere un mero strumento di validazione, ma un *oracolo intermedio* integrato in un approccio agentico che permette di arricchire il contesto dell'analisi effettuata con i risultati di uno strumento deterministico: l' LLM propone ipotesi sulla classificazione delle API osservate nel codice (source, sink, sanitizer), CodeQL verifica deterministicamente se il percorso di taint ipotizzato esiste, e l'esito arricchisce il contesto dell'agente per raffinare le ipotesi successive. La conferma finale è affidata alla generazione e all'esecuzione di un exploit (PoC), che costituisce l'unico verdetto sulla sfruttabilità reale della vulnerabilità individuata.

Questa decomposizione ha una conseguenza architettonica diretta: il task assegnato al LLM si riduce a classificazione semantica di nomi API, un compito per cui modelli di medie dimensioni (Qwen2.5-Coder da 14 miliardi di parametri, eseguibile localmente) si dimostrano adeguati. La complessità sintattica di CodeQL è interamente delegata a tool MCP deterministici, testabili indipendentemente dal modello.

Sibyl opera quindi in un regime completamente locale, senza dipendenze da API cloud, con garanzie di riproducibilità e tutela della riservatezza del codice analizzato.

Pattern di sicurezza

Sibyl non implementa il **Dual LLM Pattern** classico, ma il livello di indirectione introdotto dai tool MCP, che convertono codice sorgente non fidato in output strutturati prima che questi raggiungano il LLM, riduce la superficie di attacco per prompt injection in modo analogo.

La separazione di privilegio è architettonica in questo caso: l'orchestratore ha visibilità completa (repo, findings, manifests), gli agenti Detect/Validate/Verify hanno accesso scoped solo agli artefatti che gli vengono passati. Soprattutto, il codice sorgente non fidato non arriva mai grezzo al contesto LLM degli agenti, ma passa sempre attraverso CodeQL (deterministico) → SARIF → findings manifests.

Una prompt injection nascosta in un commento nel codice deve sopravvivere al parsing di CodeQL e alla conversione in SARIF strutturato prima di raggiungere l'agente: in pratica è neutralizzata.

Code-Then-Execute Pattern L'agente Verify genera il codice exploit ma lo esegue in un container Docker con `--network none`. La separazione tra generazione ed esecuzione, con l'esecuzione in sandbox isolata, è esattamente il vantaggio del pattern: anche se l'agente fosse manipolato, il comportamento imprevisto è contenuto dentro il container che poi viene distrutto (`--rm`).

I contributi principali di questo lavoro sono:

- (i) una pipeline neuro-simbolica per il rilevamento di vulnerabilità su codice Python e C basata interamente su modello locale, senza dipendenze da API cloud;
- (ii) un'architettura MCP basata su template che incapsula la complessità sintattica di CodeQL, riducendo il task del LLM a classificazione semantica di nomi API e rendendo i tool verificabili indipendentemente dal modello;
- (iii) un ciclo agentico in cui CodeQL opera come oracolo intermedio integrato con conferma dinamica tramite esecuzione di exploit in sandbox isolata;

Inoltre la decomposizione proposta rende il task trattabile per un modello da 14B parametri su hardware consumer, abbassando la barriera di adozione rispetto agli approcci esistenti basati su LLM commerciali.

Metodologia

Sibyl organizza l'analisi in una pipeline a tre fasi sequenziali, ciascuna eseguita da un agente autonomo che opera su un artefatto prodotto dalla fase precedente.

La prima fase, **Detect**, si occupa di individuare ipotesi di vulnerabilità. L'agente riceve il repository target e il database CodeQL precompilato, e utilizza i tool MCP per eseguire query — sia query ufficiali della libreria CodeQL che query generate on-the-fly a partire dalla superficie API del codice. L'output è una lista di findings con stato *todo*: percorsi di taint candidati, non ancora confermati.

La seconda fase, **Validate**, affina i risultati eliminando i falsi positivi. L'agente riceve i findings e ri-esegue l'analisi di taint con ipotesi più precise sulla classificazione delle API — source, sink, sanitizer — inferite dal LLM a partire dal contesto del codice. CodeQL non è qui un giudice, ma un oracolo che risponde a domande specifiche: *questo percorso esiste? viene bloccato da questo sanitizer?* I findings vengono promossi a *confermati* o scartati con motivazione.

La terza fase, **Verify**, produce la prova definitiva. Per ogni vulnerabilità confermata, l'agente genera un Proof-of-Concept e lo esegue in una sandbox

Docker isolata. Il successo dell'exploit — verificato tramite un success oracle specifico per famiglia CWE — è l'unico verdetto sulla sfruttabilità reale.

Generazione ed esecuzione sono deliberatamente separate: il LLM produce il codice, la sandbox lo esegue con rete disabilitata e ciclo di vita effimero.

Tra le fasi, gli agenti si scambiano solo manifest leggeri, path e summary degli artefatti, mentre i dati pesanti (SARIF, log, exploit) restano su disco. Il contesto LLM si azzera ad ogni handoff, mantenendo il carico cognitivo per agente controllato.

Modello di sviluppo

Sibyl adotta un ciclo di vita a spirale con prototipi incrementali: ogni iterazione aggiunge un componente funzionante e verificabile, mantenendo il sistema eseguibile in ogni momento. L'iterazione 0 (`v0.1-prototype`) costituisce la baseline — un singolo agente di detection con LLM locale e tool MCP — e funge da punto di riferimento per le iterazioni successive. Ogni nuova iterazione integra un agente o una capacità, estendendo il sistema per composizione senza modificare il core esistente.

Iterazione	Contenuto	Tag
iter 0	Prototipo singolo agente (detection)	<code>v0.1-prototype</code>
iter 1	Agente Validate + <code>run_official_query</code>	<code>v0.2</code>
iter 2	Agente Verify + sandbox Docker + success oracle	<code>v0.3</code>
iter 3	Orchestratore + work-list + concorrenza adattiva	<code>v0.4</code>
iter 4	<code>VulnFamilyProfile</code> per famiglia CWE + backend E2B	<code>v0.5</code>

Il modello a spirale è motivato da due vincoli pratici: l'inferenza LLM locale è il collo di bottiglia computazionale, quindi la pipeline sequenziale è la configurazione naturale per hardware consumer; e la testabilità dei tool indipendentemente dal modello consente di validare ogni iterazione senza richiedere GPU, abbassando la barriera di verifica ad ogni incremento.

1. Analisi dei Requisiti

1.1 Requisiti funzionali - iter 0

ID	Requisito
FR1	Enumerare i file sorgente di una repo target (cartella o <code>.zip</code>).
FR2	Costruire un database CodeQL dalla target.
FR3	Eseguire la suite di sicurezza standard CodeQL (copertura ampia).
FR4	Offrire check curati per-CWE (taint + crypto) a singola chiamata.
FR5	Offrire query taint name-based a template parametrizzate dall'agente (<code>run_taint_query</code>).
FR6	Offrire query crypto/API-misuse name-based a template (<code>run_api_misuse_query</code>).
FR7	Mantenere una knowledge base CWE (MITRE ufficiale + azioni) interrogabile (<code>list_cwes</code> , <code>cwe_knowledge</code>).
FR8	Orchestrare un LLM Ollama locale via tool-calling MCP (evidence-first).
FR9	Ancorare ogni CWE riportato ai tag SARIF deterministici (no CWE allucinati).
FR10	Produrre un report Markdown con header di metadati (modello, durata, tool, CWE, findings), organizzato per repo.
FR11	Riprendere un'analisi interrotta da checkpoint.
FR12	Robustezza: recupero tool call scritte come testo; fix del <code>db_path</code> placeholder; anti-loop.
FR13	Estendere la KB via downloader MITRE + generatore di azioni.

1.2 Requisiti funzionali - iter 1-4

ID	Requisito
FR14	Detection con query ufficiali wrappate (<code>run_official_query</code>), selezione per-CWE dal wiki.
FR15	Agente Validate : riconferma il taint-path e scarta falsi positivi (sanitizer).
FR16	Agente Verify : genera un exploit per la famiglia CWE e lo esegue in sandbox isolata.
FR17	Success oracle per famiglia (segfault/ASAN, eccezione, dati esfiltrati).

ID	Requisito
FR18	Backend sandbox intercambiabile (<code>docker</code> locale / <code>e2b</code> remoto) via config.
FR19	Work-list con <code>status</code> per ripresa , idempotenza e handoff tra agenti.
NFR9	Portabilità : dipendenze d'ambiente isolate in config; eseguibile in locale (Docker) o cloud (E2B) cambiando un flag.
NFR10	Concorrenza adattiva : sequenziale su CPU, fan-out su GPU, senza modifiche al core.
NFR11	Testabilità : i tool sono validabili senza LLM (no GPU/RAM); l'e2e con modello è raro.
NFR12	Isolamento : l'exploit gira con rete disabilitata e container effimero.

1.2 Requisiti non funzionali

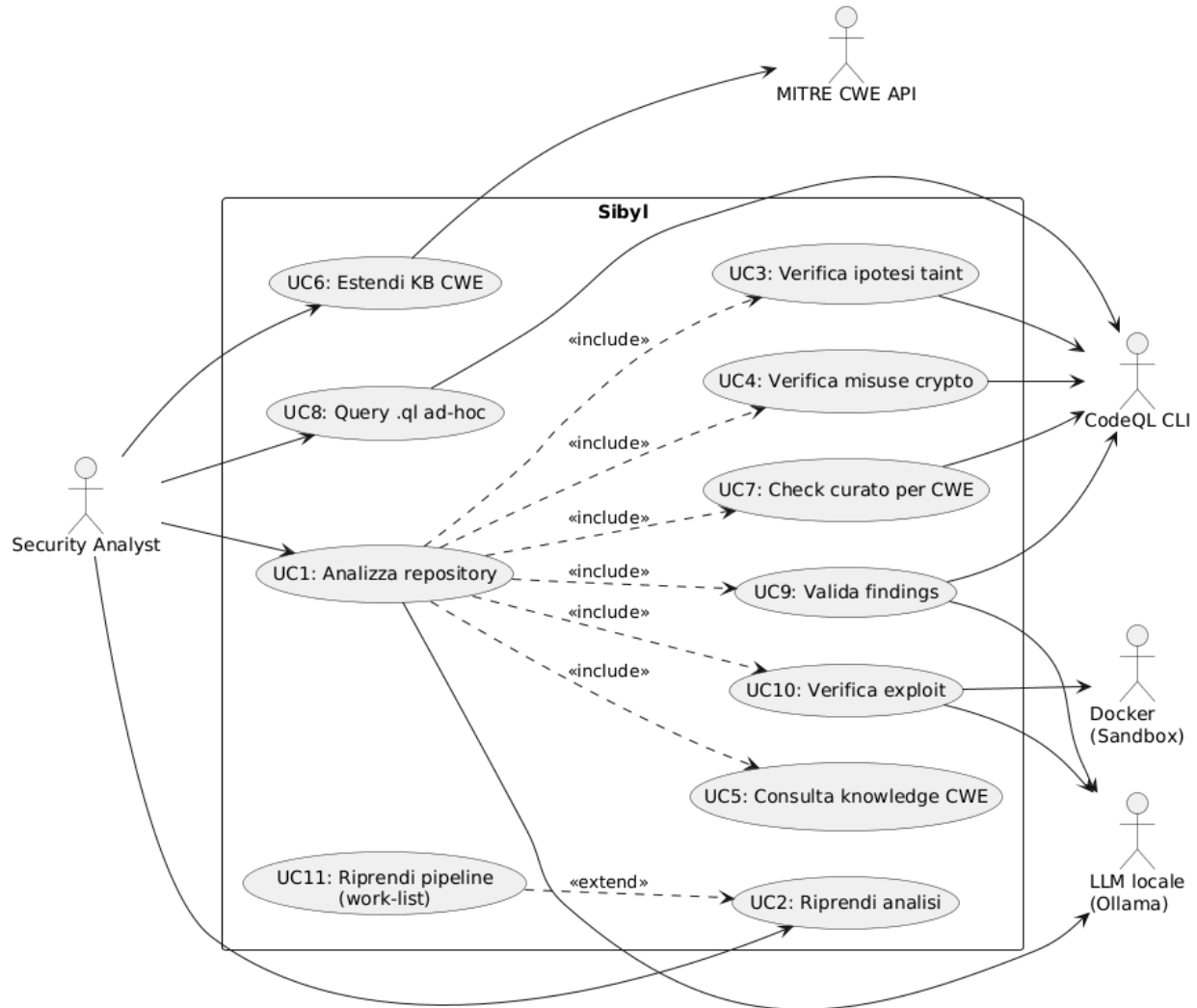
ID	Requisito
NFR1	Local/offline: nessun download di pack; query dal checkout locale <code>vscode-codeql-starter</code> .
NFR2	Verifica deterministica (CodeQL); l'LLM solo orchestra/interpreta.
NFR3	Iniezione sicura: validazione dei nomi (regex) impedisce rotture/injection in QL.
NFR4	Resilienza allo spegnimento (checkpoint, scrittura atomica).
NFR5	Model-agnostic: funziona con modelli locali deboli grazie ai fallback.
NFR6	Riproducibilità: metadati di run nel report.
NFR7	Estensibilità: tool da registry, KB in JSON, template.
NFR8	Performance: evidence-first per minimizzare gli step; riuso del DB.

2. Use Case

Attori: Security Analyst, LLM locale (Ollama), CodeQL CLI, MITRE CWE API, Docker (Sandbox).

ID	Use case	Descrizione
UC1	Analizza repository	L'analista lancia l'agente su una repo → ottiene un report.

ID	Use case	Descrizione
UC2	Riprendi analisi	Riprende una run interrotta da checkpoint (<code>-- resume</code>).
UC3	Verifica ipotesi taint	Conferma un flusso source → sink con la query templated.
UC4	Verifica misuse crypto	Rileva API crittografiche deboli (point detection).
UC5	Consulta knowledge CWE	Legge la pagina wiki (ufficiale + azioni) di una classe.
UC6	Estendi KB CWE	Scarica da MITRE + genera le azioni per nuovi CWE.
UC7	Check curato per CWE	Esegue un check fisso via query ufficiale o template.
UC8	Query .ql ad-hoc	Esegue un file <code>.ql</code> arbitrario.
UC9	Valida findings	L'agente Validate riconferma il taint-path e scarta falsi positivi.
UC10	Verifica exploit	L'agente Verify genera un PoC e lo esegue in sandbox Docker.
UC11	Riprendi pipeline	Riprende da un checkpoint della work-list multi-agente (<code>status</code> parziale).



3. Sequence Diagram — analisi di una repo (UC1)

```

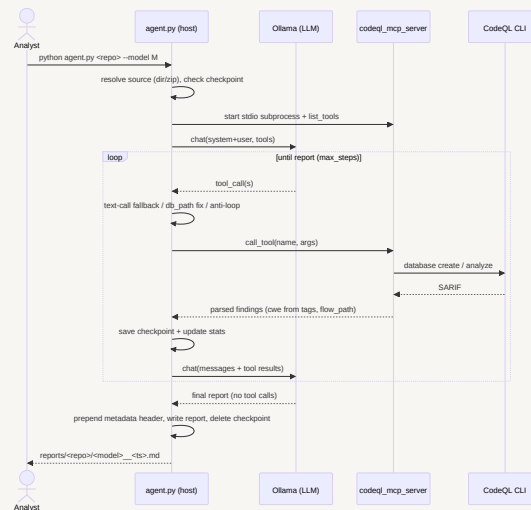
sequenceDiagram
    actor Analyst
    participant Agent as agent.py (host)
    participant LLM as Ollama (LLM)
    participant MCP as codeql_mcp_server
    participant CQL as CodeQL CLI

    Analyst->>Agent: python agent.py <repo> --model M
    Agent->>Agent: resolve source (dir/zip), check checkpoint
    Agent->>MCP: start stdio subprocess + list_tools
    Agent->>LLM: chat(system+user, tools)
    loop until report (max_steps)
  
```

```

LLM-->>Agent: tool_call(s)
Agent->>Agent: text-call fallback / db_path fix / anti-loop
Agent->>MCP: call_tool(name, args)
MCP->>CQL: database create / analyze
CQL-->>MCP: SARIF
MCP-->>Agent: parsed findings (cwe from tags, flow_path)
Agent->>Agent: save checkpoint + update stats
Agent->>LLM: chat(messages + tool results)
end
LLM-->>Agent: final report (no tool calls)
Agent->>Agent: prepend metadata header, write report, delete checkpoint
Agent-->>Analyst: reports/<repo>/<model>__<ts>.md

```



Architettura

Evoluzione del sistema

La baseline (**v0.1-prototype**) è un singolo agente di detection: LLM Ollama locale che orchestra via tool-calling MCP un insieme di tool che incapsulano CodeQL, producendo un report di findings. Le iterazioni successive aggiungono un agente per volta mantenendo il core invariato ed estendendo per composizione. Il principio guida è Open/Closed: ogni asse di variazione — linguaggio, famiglia CWE, backend sandbox — vive in un solo punto.

Pipeline a tre agenti

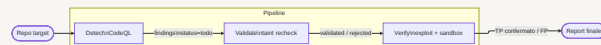
Il sistema target organizza l'analisi in una pipeline sequenziale **Detect** → **Validate** → **Verify**, comunicante tramite work-list con campo **status** . Ogni agente consuma

l'artefatto prodotto dalla fase precedente e ne aggiorna lo stato. Gli artefatti pesanti (SARIF, exploit, log ASAN) restano su disco; tra agenti transitano solo manifest leggeri con path e summary (**Claim Check pattern**). Il contesto LLM si azzerava ad ogni handoff, mantenendo controllato il carico per agente.

```

flowchart LR
    R([Repo target]) --> D
    subgraph PIPE[Pipeline]
        D[Detect\nCodeQL] -->|findings\nstatus=todo| V[Validate\nntaint recheck]
        V -->|validated / rejected| X[Verify\nexploit + sandbox]
    end
    end
    X -->|TP confermato / FP| REP([Report finale])

```



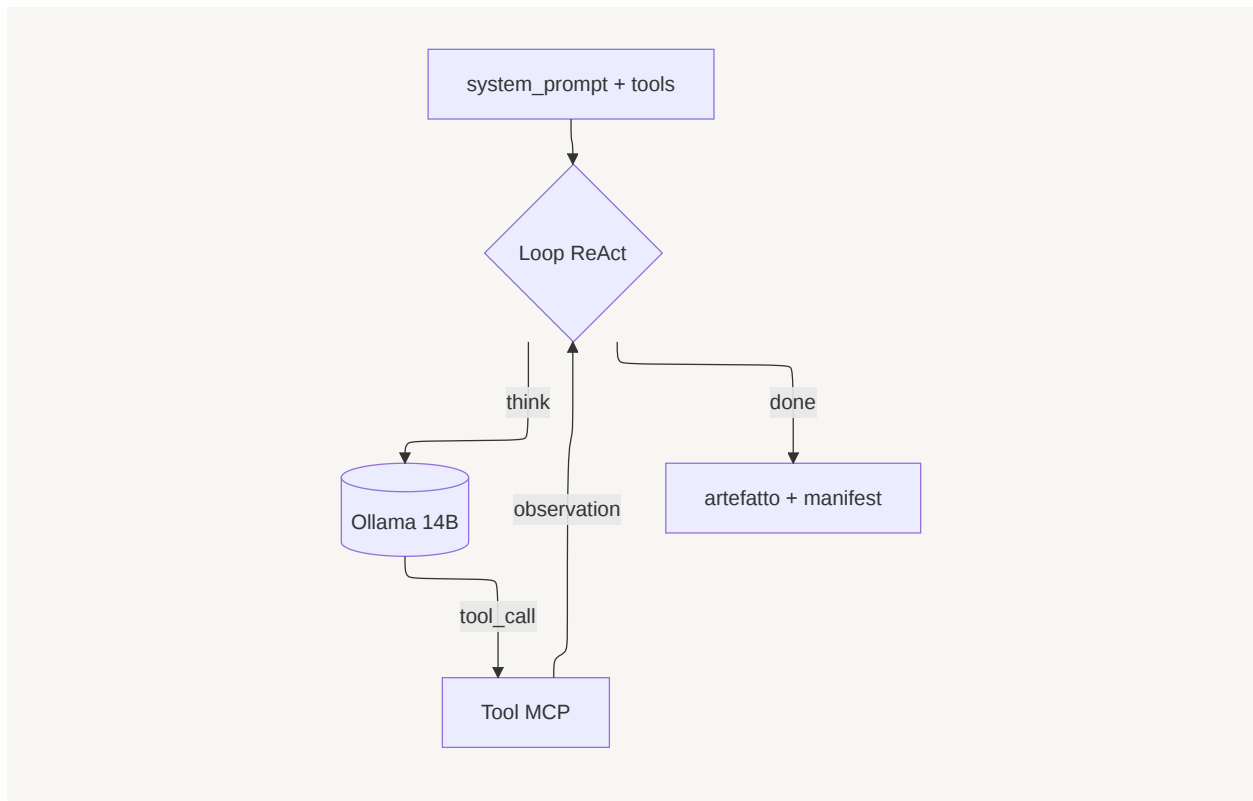
Loop agentico — ReAct

Un unico loop ReAct, parametrizzato da `system_prompt` + `tools[]`, viene riutilizzato per tutti e tre gli agenti (**Template Method**). Ad ogni iterazione il LLM ragiona sul contesto corrente, emette una tool call, riceve l'observation e aggiorna il proprio stato fino a produrre l'artefatto finale. I tool MCP sono deterministici e testabili indipendentemente dal modello (NFR11).

```

flowchart TB
    S[system_prompt + tools] --> L{Loop ReAct}
    L -->|think| LLM[Ollama 14B]
    LLM -->|tool_call| T[Tool MCP]
    T -->|observation| L
    L -->|done| OUT[artefatto + manifest]

```



Tool MCP e template system

L'agente non scrive mai CodeQL direttamente. Ogni modalità di detection corrisponde a un tool MCP che incapsula un template parametrizzato:

Tool	Categoria	Descrizione
<code>list_python_files</code>	Repo	Enumera i file sorgente di una repo (cartella o <code>.zip</code>).
<code>create_codeql_database</code>	Analisi	Costruisce il database CodeQL dalla repo target.
<code>analyze_database</code>	Analisi	Esegue la suite di sicurezza standard CodeQL.
<code>run_custom_query</code>	Analisi	Esegue un file <code>.ql</code> arbitrario sul database.
<code>run_taint_query</code>	Template	Genera ed esegue una query taint name-based (source/sink/sanitizer parametrizzati).

Tool	Categoria	Descrizione
<code>run_api_misuse_query</code>	Template	Genera ed esegue una query per API-misuse e crypto debole.
<code>run_insecure_config_flag_query</code>	Template	Rileva flag di configurazione insicuri.
<code>run_official_query</code>	Libreria	Esegue una query ufficiale del checkout <code>vscode-codeql-starter</code> via <code>--search-path</code> .
<code>list_official_queries</code>	Libreria	Elenca le query ufficiali disponibili per CWE.
<code>extract_api_surface</code>	Ipotesi	Estrae le API che ricevono dati non attendibili (stage 1 dell'inferenza).
<code>run_inspection_query</code>	Ipotesi	Lista source/sink/sanitizer che CodeQL riconosce per un dato CWE.
<code>run_hypothesis_query</code>	Ipotesi	Esegue una query taint con specifiche LLM-inferred e <code>isBarrier</code> come oracolo.
<code>list_cwes</code>	Knowledge base	Elenca tutti i CWE presenti nella KB.
<code>cwe_knowledge</code>	Knowledge base	Restituisce la pagina wiki (MITRE + azioni) per un CWE.
<code>read_file_snippet</code>	Utilità	Legge un frammento di file sorgente per linea.

Il LLM fornisce i nomi delle API; i template costruiscono la query QL. I nomi vengono validati tramite regex prima dell'inserimento (NFR3), impedendo injection nel DSL. La selezione del tool per CWE è codificata in `cwe_wiki.json` e restituita da `cwe_knowledge` — il LLM non decide la strategia di analisi, la riceve già pronta.

VulnFamilyProfile

Ogni famiglia CWE espone un profilo che astrae all'agente: quale tool usare, il prompt per step, i tool aggiuntivi e il **success oracle** — la condizione che definisce exploit riuscito: segfault/ASAN per vulnerabilità di memoria in C, eccezione o dati esfiltrati per injection in Python. L'ereditarietà `base → classe → famiglia` evita duplicazione (**Facade + Strategy**).

Sandbox e isolamento

L'agente Verify genera il codice exploit ma non lo esegue direttamente: lo consegna al tool `run_exploit`, che lo esegue in un container Docker con `--network none` e `--rm`. Generazione ed esecuzione sono deliberatamente separate — il LLM non ha accesso alla shell del container. Il success oracle valuta exit code, stderr e pattern nei log, restituendo all'agente un risultato strutturato. Su Windows l'ambiente C/ASAN è esposto tramite WSL2.

Pattern architetturali

Componente	Pattern	Asse di variazione
Orchestratore	Mediator + Pipeline	sequenza degli step
Agent loop	ReAct + Template Method	prompt + tool per fase
Tool MCP	Plugin / Registry	capacità per agente
Adapter linguaggio	Strategy + Adapter	Python / C
VulnFamilyProfile	Facade + Strategy	famiglia CWE
Sandbox backend	Strategy	Docker
Artifact store	Blackboard + Claim Check	comunicazione tra agenti
Work-list	Worklist + Handoff	findings con <code>status</code>

Component diagram

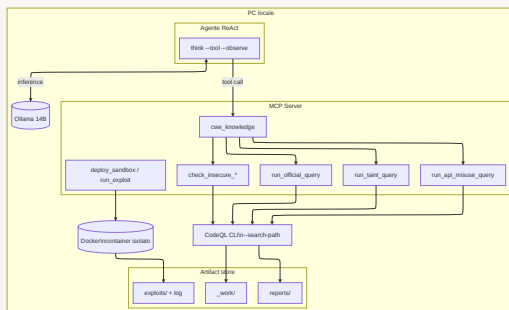
```
flowchart TB
    subgraph PC["PC locale"]
        M["(Ollama 14B)"]
        subgraph AG["Agente ReAct"]
            LOOP["think→tool→observe"]
        end
    end
    subgraph MCP["MCP Server"]
        KB["cwe_knowledge"]
        OFF["run_official_query"]
        TNT["run_taint_query"]
        API["run_api_misuse_query"]
        CFG["check_insecure_*"]
        SBX["deploy_sandbox / run_exploit"]
    end
    CQL["CodeQL CLI\n--search-path"]
```

[e373c872-84a3-4b1b-8550-ee1554ecdf10_ExportBlock-aeb976ff-6f40-4fc8-b9af-54dc5793a934.zip](#)

```

DOCK[(\docker\container isolato)]
subgraph STORE[Artifact store]
  DB[_work/]
  REP[reports/]
  EXP[exploits/ + log]
end
end
end
LOOP <-->|inference| M
LOOP -->|tool call| KB --> OFF & TNT
& API & CFG
OFF & TNT & API & CFG --> CQL --> DB
& REP
SBX --> DOCK --> EXP

```



Sequence diagram — pipeline 3 agenti

```

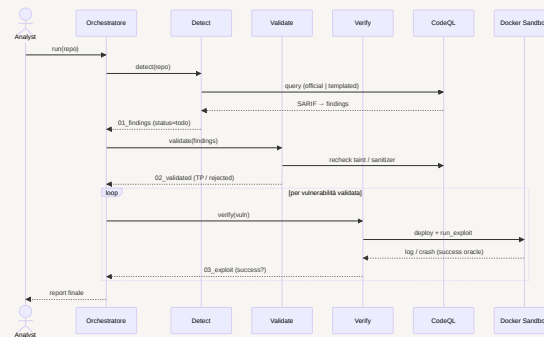
sequenceDiagram
    actor Analyst
    participant Orch as Orchestratore
    participant Det as Detect
    participant Val as Validate
    participant Ver as Verify
    participant CQL as CodeQL
    participant SBX as Docker Sandbox

    Analyst->>Orch: run(repo)
    Orch->>Det: detect(repo)
    Det->>CQL: query (official | templated)
    CQL-->>Det: SARIF → findings
    Det->>Orch: 01_findings (status=todo)
    Orch->>Val: validate(findings)
    Val->>CQL: recheck taint / sanitizer
    Val-->>Orch: 02_validated (TP / rejected)
    loop per vulnerabilità validata
        Orch->>Ver: verify(vuln)
        Ver->>SBX: deploy + run_exploit
        SBX-->>Ver: log / crash (success oracle)
        Ver-->>Orch: 03_exploit (success?)
    end

```

end

Orch-->>Analyst: report finale



Passiamo alla strategia di test o hai altro da aggiungere qui?

Class / Component Diagram

Il progetto è prevalentemente a moduli + funzioni; i moduli sono rappresentati come componenti e le strutture dati come classi.

```
classDiagram
    class Agent {
        <<module agent.py>>
        +SYSTEM_PROMPT
        +run_agent(repo, model, report, max_steps, resume)
        +main()
        -mcp_tools_to_ollama(tools)
        -_extract_text_tool_calls(content, names)
        -_plain(msg)
        -_checkpoint_path(repo)
        -_save_checkpoint(path, state)
        -_default_report_path(repo, model)
        -_metadata_header(...)
        -_resolve_source(path)
    }
    class CodeqlMcpServer {
        <<FastMCP server>>
        +list_python_files()
        +create_codeql_database()
        +analyze_database()
        +run_custom_query()
        +run_taint_query()
        +run_api_misuse_query()
        +list_cwes()
        +cwe_knowledge()
        +read_file_snippet()
        -_analyze_with_query()
    }
```

```

    -_parse_sarif()
    -_extract_flow()
    -_cwe_from_tags()
    -_render_names()
    -_render_consts()
    -_normalize_cwe()
}

class Config {
    <<module config.py>>
    +AGENT_MODEL
    +CODEQL_SEARCH_PATH
    +DEFAULT_SUITE
    +CUSTOM_QUERIES
    +TEMPLATE_DIR
    +CWE_WIKI_PATH
    +WORK_DIR
    +REPORTS_DIR
}

class KbTools {
    <<fetch_mitre.py / build_actions.py>>
    +fetch_mitre.main()
    +build_actions.main()
}

class Finding {
    +rule_id
    +cwe
    +file
    +line
    +source
    +sink
    +flow_path
}

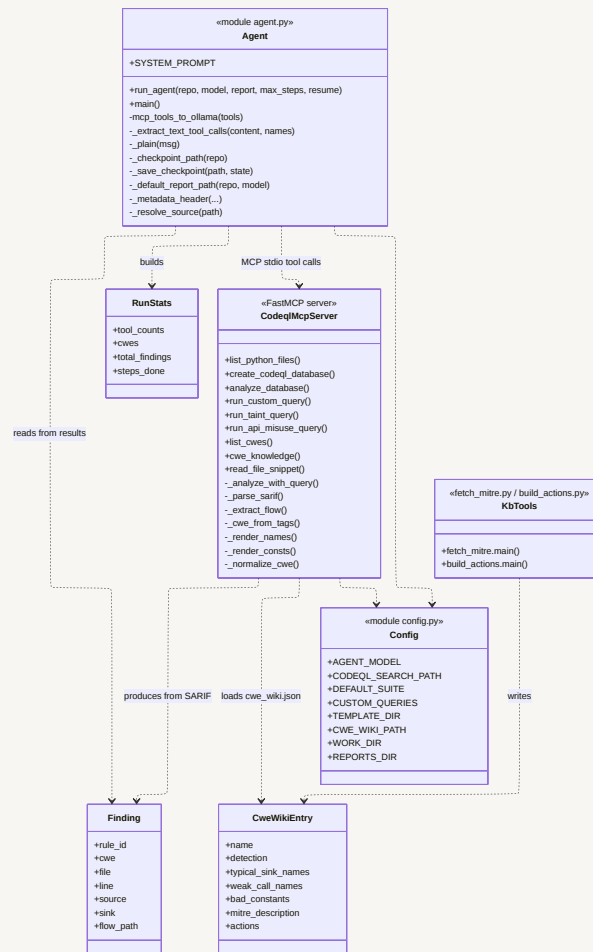
class CweWikiEntry {
    +name
    +detection
    +typical_sink_names
    +weak_call_names
    +bad_constants
    +mitre_description
    +actions
}

class RunStats {
    +tool_counts
    +cwes
    +total_findings
    +steps_done
}

Agent ..> CodeqlMcpServer : MCP stdio tool calls
Agent ..> Config
Agent ..> RunStats : builds
Agent ..> Finding : reads from results
CodeqlMcpServer ..> Config

```

CodeqlMcpServer ..> Finding : produces from SARIF
 CodeqlMcpServer ..> CweWikiEntry : loads cwe_wiki.json
 KbTools ..> CweWikiEntry : writes



5. Package Diagram

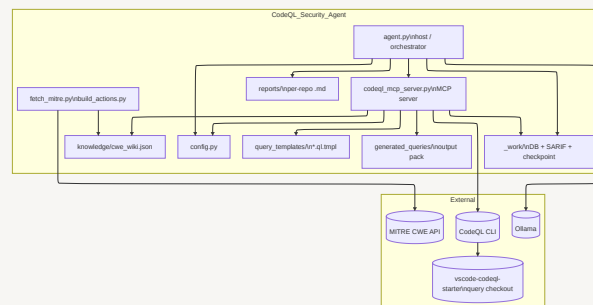
```

flowchart TB
    subgraph PKG[CodeQL_Security_Agent]
        agent[agent.py\nhost / orchestrator]
        server[codeql_mcp_server.py\nMCP server]
        config[config.py]
        kb[fetch_mitre.py\nbuild_actions.py]
        templates[query_templates\n*.ql.tpl]
        wiki[knowledge/cwe_wiki.json]
        gen[generated_queries\noutput pack]
        work[_work\nDB + SARIF + checkpoint]
        reports[reports\nper-repo .md]
    end
    subgraph EXT[External]
    end
  
```

```

ollama[(Ollama)]
codeql[(CodeQL CLI)]
starter[(vscode-codeql-starter\nquery checkout)]
mitre[(MITRE CWE API)]
end
agent --> server
agent --> config
agent --> reports
agent --> work
agent --> ollama
server --> config
server --> templates
server --> wiki
server --> gen
server --> work
server --> codeql
codeql --> starter
kb --> wiki
kb --> mitre

```



- agent builder
 - il flow mi da tutte le funzioni sui diversi file
- i taint devono essere effettivamente esposti dalle api per essere user controlled